

Conversational Refactor — Assessment

Turning the five module screens into one continuous chat workspace, plus the round-2 testing items

Date: 2026-09-22 • Repository: DRP-UI, branch **main** • Status: **assessment only — no code changed**

Purpose

This document answers one question: **what has to change, and in which files**, to make the app behave like one continuous AI conversation instead of five separate screens. It also covers the frontend items from the latest testing round. Nothing here has been implemented — this is for review first, as requested.

Headline finding

The architecture is already right. The problem is almost entirely in one function.

The session store in `src/hooks/useWorkflowSession.js` already keeps the conversation as a single append-only thread, tagged per module, and it is never pruned. The comment at the top of that file says so explicitly: *“the conversation is a single append-only thread whose entries are tagged with the module they belong to. Navigating back to an earlier step changes which slice is shown, never what is stored.”* Per-module state, job polling, the module registry, phase resolution and the API layer all survive this refactor untouched.

What makes it *feel* like five screens is that `renderContent()` in `CompleteWorkflow.jsx:1967` returns exactly one phase component at a time, and each phase component draws its own private chat bubbles. The thread exists; it is simply never shown as one thread.

The five bugs behind “it feels broken”

Before the big refactor, five specific defects explain most of what the testers hit. Each is small and independent. **Fixing these gets most of the conversational feel for a fraction of the work**, and they are worth doing first whatever is decided about the full shell.

#	Defect	Where
B1	Chat replies are invisible on three of the five modules. <code>chatMessages</code> is passed only to <code>LitMineX</code> and <code>CurateX</code> . <code>TxKG</code> , <code>ScreenSuite</code> and <code>NovSearch</code> never render the thread, so a follow-up asked on those screens is stored in state and never drawn. This is exactly the tester's “a JSON response is received but no answer is displayed”.	<code>CompleteWorkflow.jsx</code> 2061, 2091 (and the three that omit it: 2032, 2124, 2138)
B2	The composer is hidden entirely on NovSearch. The render guard is <code>viewMode === "chat" && !workflowPhase.startsWith("novelty")</code> , so the conversation dead-ends at the last module.	<code>CompleteWorkflow.jsx:2202</code>
B3	Replies with no recognised content field are dropped silently. <code>sendMessage</code> appends the agent message only if <code>(parsed.content)</code> . If the response envelope differs even slightly, the user sees nothing at all — no message, no error.	<code>useWorkflowSession.js:597</code> <code>moduleMap.js:388</code>
B4	History is truncated when you navigate back. <code>visibleConversation</code> filters to <code>stepIndex <= activeModule.index</code> , so stepping back to <code>TxKG</code> hides everything discussed later. The opposite of a continuous thread.	<code>useWorkflowSession.js:689-694</code>

#	Defect	Where
B5	There is no way to start a new disease query in the same workspace. startSession runs once on mount from location.state.query, and the chat bar only ever POSTs to /messages. A second disease has nowhere to go.	CompleteWorkflow.jsx:205-222 CompleteWorkflow.jsx:2178

B1, B2 and B3 together fully explain the reported symptom. B5 explains why one session only ever covers one disease on the frontend side — note the backend also has its own version of this problem, which is out of scope here.

Part A — The conversational refactor

A.1 What stays exactly as it is

The brief says preserve the architecture. That is achievable: none of the following needs to be touched, and the refactor should be reviewed against this list.

Layer	File	Why it survives
API client	src/services/apiClient.js	Auth, interceptors, 401 handling. No change.
Service layer	src/services/api/*.js	sessions, jobs, txkg, litminex, curatex, screensuite, novsearch, exports, modules. Every endpoint and payload shape stays.
Session store	src/hooks/useWorkflowSession.js	Reducer, per-module steps, activation order, append-only conversation. Two small derived-value changes only (B4, W3).
Module registry	src/workflow/moduleMap.js	Keys, apiKeys, phase prefixes, parsers. Display formatting changes (T5); the registry itself does not.
Job polling	src/hooks/useJob.js	Unchanged.
Result normalisers	src/workflow/phaseResults.js, txkgResult.js, selections.js	Unchanged.
Composer	src/components/workflow/ChatInputBar.jsx	Already the right component: module-level, owns its own draft, @mention autocomplete against GET /modules/search. Becomes the single composer.

A.2 The one structural change

Today renderContent() (CompleteWorkflow.jsx:1967-2162) is a chain of phase tests that returns a single full-height component:

```
if (workflowPhase.startsWith("pipeline")) return <PipelinePhase .../>;
if (["txkg-loading", "txkg-results", ...]) return <TXKGPhase .../>;
if (["litminex-loading", ...]) return <LitminexPhase .../>;
if (["curatex-loading", ...]) return <CuratexPhase .../>;
if (workflowPhase.startsWith("screensuite")) return <ScreeningSuitePhase .../>;
if (workflowPhase.startsWith("novelty")) return <NoveltySearchPhase .../>;
```

It becomes a **timeline renderer**: walk session.conversation in order and, for each entry, draw either a message bubble or — where the entry marks a module reaching its results phase — that module's result card. The same five components render the cards; they are no longer mutually exclusive full screens.

The phase tests above do not disappear. They move inside a small that picks which component to draw for one module's current phase. The prop lists at lines 2032–2147 transfer verbatim — that is the cheapest way to keep

every existing data flow intact.

A.3 Work items

ID	Change	Files	Size
W1	Conversation timeline component. New <code>ConversationTimeline.jsx</code> that maps <code>session.conversation</code> to bubbles, in order, with the whole thread always visible. Auto-scroll on append, preserved scroll position when the user scrolls up.	new: <code>src/components/workflow/ConversationTimeline.jsx</code>	M
W2	Module result card wrapper. New <code>ModuleResultCard.jsx</code> holding the phase-to-component mapping lifted out of <code>renderContent()</code> . Collapsible, with the module name in the header. Existing prop lists carried over unchanged.	new: <code>src/components/workflow/ModuleResultCard.jsx</code> from: <code>CompleteWorkflow.jsx</code> :196-2162	M
W3	Stop truncating the thread. <code>visibleConversation</code> returns the full conversation; per-module slicing stays available through the existing <code>messagesForModule(key)</code> for the side panel. (Fixes B4.)	<code>src/hooks/useWorkflowSession.js</code> :689-694	S
W4	Phase screens become cards. Each of the five loses its outer full-height scroll container and its own chat bubbles, keeping only the result body. The internal stage machines stay — NovSearch's results/comparison/decision/compiling/summary stages become states of its card.	<code>TXKGPhase.jsx</code> , <code>LiteminexPhase.jsx</code> , <code>CuratexPhase.jsx</code> , <code>ScreeningSuitePhase.jsx</code> , <code>NoveltySearchPhase.jsx</code>	L
W5	One composer, always present. Remove the NovSearch exclusion and the <code>viewMode</code> coupling so the composer is pinned to the workspace at every phase. (Fixes B2.)	<code>CompleteWorkflow.jsx</code> :2202-2217	S
W6	New-query intent in-session. Give the composer a path for “a different disease” that goes through the supervisor rather than only <code>/messages</code> — either <code>POST /sessions</code> for a fresh session, or a TxKG hand-off carrying the new disease, whichever the backend prefers. Needs a backend decision before building. (Addresses B5 on the UI side.)	<code>CompleteWorkflow.jsx</code> :2178-2184 <code>useWorkflowSession.js</code> :541-629	M
W7	Never drop a reply. When a message response carries no recognised content, render a visible fallback instead of nothing, and log the raw envelope. (Fixes B3.)	<code>useWorkflowSession.js</code> :594-610	S
W8	Rail becomes a progress indicator, not a router. <code>WorkflowStepper</code> (<code>CompleteWorkflow.jsx</code> :536) keeps showing state but scrolls the timeline to that module's card instead of swapping screens. <code>session.rail</code> already supplies everything needed.	<code>CompleteWorkflow.jsx</code> :536-...	S
W9	Results side panel. Optional but recommended: a collapsible panel listing each module's latest result, so a long thread stays navigable. Reads <code>session.steps</code> and <code>messagesForModule</code> ; no new state.	new component	M

Size: S = under a day, M = one to three days, L = three days or more, per item, including manual verification.

A.4 Risks

Risk	Mitigation
W4 is the large one. The five phase files total roughly 6,400 lines and each assumes it owns the viewport.	Do them one at a time, starting with TxKG. Keep <code>renderContent()</code> behind a flag until all five are converted, so the old path still works during the transition.
Rendering every result card at once will be slow. The TxKG subgraph and the docking views are heavy.	Cards collapsed by default except the most recent; mount the body only when expanded.
Scroll behaviour. A job polling once a second re-renders the parent, which can fight the user's scroll position.	Same discipline already applied to <code>ChatInputBar</code> : keep scroll state in the timeline component, not the polling parent.
W6 depends on the backend. Whether a new disease means a new session or a new step is not a UI decision.	Settle it with the backend team before W6 starts. W1–W5 and W7–W9 do not depend on the answer.

A.5 Suggested order

Phase 1 — the five bugs (B1, B2, B3, B4, B5 partial). W3, W5, W7 plus passing `chatMessages` to the three screens that lack it. Small, low-risk, and it makes the app feel markedly more continuous immediately. Worth shipping on its own.

Phase 2 — the shell. W1, W2, W8 behind a feature flag, with the old render path still reachable.

Phase 3 — the cards. W4, one module at a time.

Phase 4 — polish. W9, and W6 once the backend question is settled.

Part B — Round-2 testing items (frontend only)

Backend items from the testing notes are listed in Part C and excluded from the work below, as instructed.

ID	Item	File and line	Change
T1	Home query heading	<code>HomePage.jsx:340-342</code> <code>NewResearchPage.jsx:584-586</code>	“What new drug you are going to Rediscover today? ” becomes “What drug are you going to repurpose today? ”. The green <code><Box component="span"></code> highlight should carry the same trailing words as now.
T2	Remove query box sub-label	<code>HomePage.jsx:546</code> <code>NewResearchPage.jsx:1031</code>	Delete the “Describe a disease or research intent” Typography entirely. Check the surrounding flex spacing after removal — it is a <code>flexShrink: 0</code> element inside a stacked box.
T3	Welcome step 1 needs therapeutic-area boxes	<code>WelcomeScreen.jsx</code> , the <code>step-1 StepCard</code>	Add <code>tags={['Oncology', 'Rare Diseases', 'Inflammation', 'Neurodegenerative']}</code> so step 1 matches step 2's pill row. Note: this reverses review point 8 , which asked for the tags to be removed. The <code>StepCard</code> component already supports optional tags, so this is a one-line change.
T4	Signup, Help Center, Privacy, Terms	<code>Login.js</code> footer and <code>remember/forgot</code> row	Unchanged and still blocked — same as review point 4. All four controls are visible and inert. Needs the keep-or-remove decision before any work.

ID	Item	File and line	Change
T5	Module name format	moduleMap.js:232 (agentHeadingFor) AgentHeader.jsx and the 5 phase files	Current output is "TXKG – TARGET IDENTIFICATION AGENT" (uppercased, em-dash). Required is two lines: TxKG as written, with (Target Identification Agent) beneath it, smaller and dimmer. This reverses the uppercase form introduced for review point 16. See below.

B.1 T5 in detail — the module name format

Five call sites read `agentHeadingFor(key)`, which today returns an uppercased single string. The requested format is two pieces of text with different weight and colour, so a single string can no longer carry it.

Recommended approach

Keep `MODULES` exactly as it is — `label` and `agentRole` already hold the two halves, correctly cased ("TxKG", "Target Identification Agent"). Add a `moduleDisplayFor(key)` returning `{ label, role }`, and have `AgentHeader` render the two lines. Retire `agentHeadingFor` once the five call sites are converted.

Exact required strings: TxKG (Target Identification Agent), LitMineX (Literature Mining Agent), CurateX (Drug Curation Agent), ScreenSuite (Virtual Screening Agent), NovSearch (Novelty Search Agent). No all-caps anywhere.

Call sites to update: `TXKGPhase.jsx:88`, `LiteminexPhase.jsx:98`, `251`, `CuratexPhase.jsx:177`, `ScreeningSuitePhase.jsx:337`, `NoveltySearchPhase.jsx:220`. `ScreenSuite` and `NovSearch` still have their own local `AgentHeader` definitions (`ScreeningSuitePhase.jsx:302`, `NoveltySearchPhase.jsx:186`) — folding those into the shared component is worth doing at the same time.

B.2 Two reversals worth confirming

T3 and T5 both undo changes made for the previous review round. That is fine — requirements move — but it is worth a one-line confirmation from the reviewers before the work is done, so the same ground is not covered a third time.

Then	Now
Review point 8: step 1 — "remove the rest", so the three tag pills were deleted.	T3: step 1 needs four therapeutic-area boxes (Oncology, Rare Diseases, Inflammation, Neurodegenerative).
Review point 16: one uniform agent name per module; implemented as an uppercase heading.	T5: same names, but not all-caps — module name as written, full agent name smaller and dimmer beneath it.

Part C — Backend items (excluded, listed for completeness)

These came through in the same testing notes and are recorded here so nothing is lost, but no frontend work is proposed for them.

Area	Item
Supervisor routing	Any prompt — including "open report.pdf, j" — is resolved into a disease name and sent to TxKG. There is no rejection path for input that is not a research query.
Session scope	A second disease in the same session keeps returning the first disease's TxKG results. The backend answer itself says the results are for the earlier disease and suggests rerunning TxKG. The frontend half of this is B1/B3 above — the reply is received and then not rendered. Both halves need fixing.

Area	Item
Therapeutic focus	Feasibility of the research-focus area set — 3 test cases still blocked on it. Note the frontend side is now wired: GET/POST /users/me/research-focus are called from the welcome screen as of commit efec67a.
Carried over	Review points 12, 15, 25 and 26 remain open — see iReNovo_UI_Review_Open_Items.pdf in this repository.

Summary

Group	Items	Notes
Bug fixes, low risk	B1, B2, B3, B4	Small and independent. Explain most of the reported symptom. Do these first.
Refactor	W1–W5, W7–W9	Staged behind a flag. W4 is the bulk of the effort.
Blocked on backend	W6 / B5	Whether a new disease is a new session or a new step.
Copy and format	T1, T2, T3, T5	Small. T3 and T5 reverse earlier decisions — confirm first.
Blocked on product	T4	Signup / Help Center / Privacy / Terms. Same as review point 4.

Nothing in this document has been implemented. Findings B1–B5 come from reading the current main and are cited to file and line so each can be checked independently. They have not been reproduced against a running backend.